

Relatório de Auditoria de Segurança

Projeto: CycloneDX SCA Visualizer & Dependency Tree Platform (v1.7.1)

Data da Auditoria:	29 de Agosto de 2026
Classificação:	Defensivo / Revisão Estática de Segurança (SAST & Arquitetura)
Avaliador:	Security-First AI Agent (Antigravity Guardian v2.0)
Escopo Auditado:	Frontend React/TypeScript, Nginx Config, Dockerfile, CI/CD GitLab Pipelines e Documentação
Resultado Geral:	APROVADO COM RESSALVAS (4 Recomendações Acionáveis / 0 Falhas Críticas)

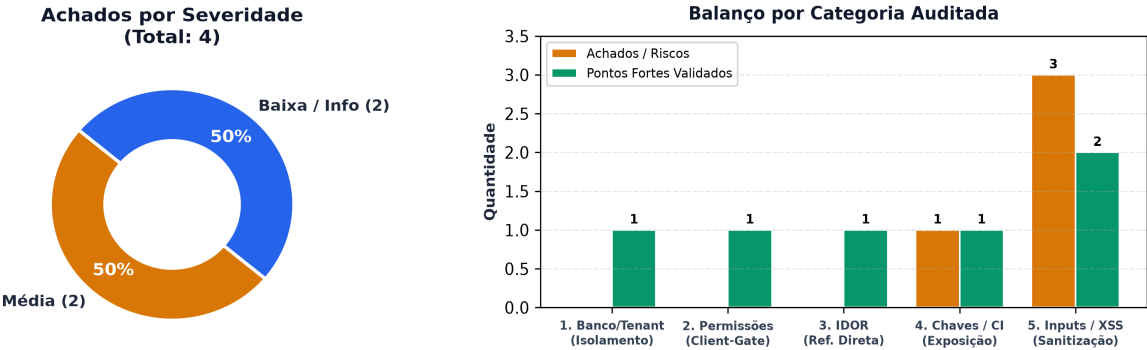
Nota Metodológica & Mapeamento de Stack

Antes do início da varredura, a pilha tecnológica completa do projeto foi identificada para adequar rigorosamente cada categoria aos mecanismos técnicos reais do ecossistema:

Camada	Tecnologia Detectada	Adaptação Metodológica
Frontend / SPA	React 18.3, TypeScript 5.7, Vite 6.1, Zustand 5.0, TailwindCSS	Auditoria de XSS, DOMPurify, injeções em grafo e parsing client-side.
Banco / Persistência	Nenhum (100% Client-Side RAM Processing)	Mapeado para isolamento de dados em memória local e ausência de vazamento.
Autenticação / RBAC	Nula / Anônima por Design (WebMCP Agent RFC Mocks)	Verificação de ausência de credenciais expostas e de gates falsos.
Deploy & Infra	Docker (Nginx Alpine Slim Hardened), GitLab CI/CD	Auditoria de headers HTTP OWASP, usuário não-root e pipelines de CI/CD.

1. Resumo Executivo

A auditoria de segurança da plataforma **CycloneDX SCA Visualizer (v1.7.1)** avaliou o código-fonte, configurações de deploy em Nginx/Docker, esteiras de DevSecOps e o fluxo de dados em memória RAM. O projeto adota uma arquitetura focada em privacidade (*Privacy-First*), realizando todo o parsing de SBOMs e cálculo de métricas localmente no navegador, sem transmissão para servidores de nuvem.



CRÍTICA	ALTA	MÉDIA	BAIXA	PONTOS FORTES
0	0	2	2	6

2. Pontos Fortes e Riscos Centrais

■ Pontos Fortes Comprovados no Código:

Controle / Proteção	Evidência Técnica no Código	Benefício de Segurança
Proteção Anti-XXE em XML	frontend/src/services/parsers/xmlParser.ts:11 processEntities: false	Elimina vetores de injeção de entidades externas e exfiltração de arquivos locais durante parsing de SBOMs XML.
Deteção de Ciclos & Anti-DoS	frontend/src/services/normalizer.ts:149-154 visited: Set<string> e depth > 32	Impede travamento do navegador (Stack Overflow / DoS) em grafos de dependências maliciosos com ciclos infinitos.
Proteção Prototype Pollution	frontend/src/context/LanguageContext.tsx:48-56, 85-93	Validação explícita de chaves __proto__, constructor e prototype no motor i18n.
Privacidade Client-Side RAM	frontend/src/store/useScaStore.ts:24-78	Zero persistência de dados em nuvem. Os arquivos SBOM nunca saem da memória RAM do navegador do usuário.
Hardening Container & Nginx	frontend/Dockerfile:31 (USER nginx) frontend/nginx.conf:21-30	Execução não-root e aplicação rigorosa de cabeçalhos HSTS, X-Frame-Options DENY, COOP, COEP e CORP.

■ Riscos Centrais Identificados:

- 1. Uso desnecessário de dangerouslySetInnerHTML:** Renderização de metadados de pacotes e sanitização aberta de links no modal.
- 2. Jobs de CI/CD não bloqueantes (allow_failure: true):** Gitleaks não interrompe a esteira em caso de segredos commitados.
- 3. Diretiva unsafe-inline no CSP:** Relaxamento da proteção contra injeções de estilo CSS inline no Nginx.

3. Tabela de Achados Detalhados por Categoria

Sev.	Categoria & Arquivo:Linha	Descrição Técnica do Achado & Risco
MÉDIA	Cat 5: Inputs sem Tratamento (XSS) frontend/src/components/explorer/Pack ageDetailModal.tsx:167, 295, 309	Uso indevido de dangerouslySetInnerHTML em dados textuais de pacotes: A linha 167 renderiza o grupo de pacotes via dangerouslySetInnerHTML interpolando a string com t('details.group'). Como o grupo é um identificador textual puro, não há justificativa técnica para renderização HTML. Além disso, a função sanitizeText (linhas 109-112) permite a tag <a> sem definir ALLOWED_ATTR explícito para rel='noopener noreferrer' e sem restringir esquemas de URI em links presentes em relatórios de CVE.
MÉDIA	Cat 4: Chaves Expostas / CI .gitlab/ci/security.gitlab-ci.yml:23, 34	Esteira de CI/CD não-bloqueante para detecção de segredos e SAST: Os jobs semgrep_sast_scan e gitleaks_secret_scan estão configurados com allow_failure: true. Caso um desenvolvedor acidentalmente comite uma chave de API privada, token de acesso ou credencial em arquivos de configuração, o pipeline continuará a execução e realizará o deploy automatizado em produção sem interromper a esteira.
BAIXA	Cat 5: Content Security Policy frontend/nginx.conf:30, 161	CSP com diretiva style-src 'unsafe-inline': O cabeçalho Content-Security-Policy aplicado nas respostas HTTP do Nginx inclui style-src 'self' 'unsafe-inline' https://fonts.googleapis.com. O uso de 'unsafe-inline' mitiga parcialmente o isolamento estrito contra injeções de atributos de estilo e exfiltração de dados via CSS em navegadores modernos.
INFO	Cat 5: Resiliência de Parser frontend/src/services/parsers/jsonParser .ts:5	Ausência de validação de tamanho de payload pré-parse JSON: O método parseJsonSbom executa diretamente JSON.parse(content). Embora seguro contra injeção de código, arquivos JSON artificiais com dezenas de megabytes enviados ao Dropzone podem levar a alto consumo de heap e congelamento temporário da UI (Client-Side DoS).

4. Recomendações Priorizadas de Remediação

Prioridade	Ação Recomendada	Impacto e Justificativa
P1 (Alta)	Eliminar dangerouslySetInnerHTML em metadados de grupo e endurecer DOMPurify Substituir a linha 167 de PackageDetailModal.tsx por renderização JSX pura <code>{t('details.group')}</code> : <code>{component.group}</code> e configurar <code>ALLOWED_ATTR</code> : <code>['href', 'target', 'rel']</code> com hook forçando <code>rel='noopener noreferrer'</code> .	Elimina qualquer risco de parsing inconsistente no DOM e bloqueia abertura insegura de links de CVEs em novas abas.
P1 (Alta)	Bloquear esteira de CI/CD na detecção de segredos (Gitleaks) Remover <code>allow_failure: true</code> do job <code>gitleaks_secret_scan</code> em <code>.gitlab/ci/security.gitlab-ci.yml</code> .	Garante que nenhum token de acesso ou segredo commitado acidentalmente seja publicado no ambiente de produção.
P2 (Média)	Refinamento de CSP no Nginx Remover <code>'unsafe-inline'</code> da diretiva <code>style-src</code> em <code>nginx.conf</code> ou migrar estilos inline para classes Tailwind utilitárias compiladas no build.	Fortalece a proteção contra injeções de estilo e CSS exfiltration.
P3 (Baixa)	Limite de tamanho de arquivo no Dropzone Adicionar validação em <code>Dropzone.tsx</code> limitando arquivos a 50MB antes da leitura pelo <code>FileReader</code> .	Previne exaustão de memória da aba do navegador em arquivos anômalos.

5. Issues para o GitHub (Prontas para Criação)

Abaixo estão os textos completos formatados em Markdown para abertura direta das issues de remediação de segurança no repositório. As issues estão prontas para cópia e cola:

```
--- ISSUE 1 ---
Título: [Segurança] Remover dangerouslySetInnerHTML em metadados de pacotes e restringir atributos no DOMPurify
Labels: security, severity:medium, bug, frontend

### ■ Descrição do Problema
Em `frontend/src/components/explorer/PackageDetailModal.tsx`, a propriedade `component.group` é renderizada utilizando `dangerouslySetInnerHTML`. Como o grupo é uma sequência alfanumérica de identificação (ex: `org.apache.commons`), não há necessidade de renderização via HTML. Além disso, a configuração do `DOMPurify.sanitize` permite a tag `` sem restringir explicitamente atributos permitidos e sem forçar `rel="noopener noreferrer"`.

### ■ Evidência no Código
Arquivo: `frontend/src/components/explorer/PackageDetailModal.tsx`
- Linha 167:
`<p className="text-xs text-slate-400 font-mono" dangerouslySetInnerHTML={{ __html: `${t('details.group')}`:
`${sanitizeText(component.group)}` }} />`
- Linhas 109-112:
`const sanitizeText = (text?: string): string => {
  `if (!text) return '';`
  `return DOMPurify.sanitize(text, { ALLOWED_TAGS: ['b', 'i', 'em', 'strong', 'code', 'p', 'br', 'a'] });`
};`

### ■ Impacto
Possibilidade de renderização de links manipulados em descrições de vulnerabilidades e risco de anomalias no parsing de strings do DOM.

### ■ Sugestão de Correção
1. Substituir a linha 167 por:
`{component.group && <p className="text-xs text-slate-400 font-mono">{t('details.group')}: {component.group}</p>}`
2. Ajustar `sanitizeText` para:
`DOMPurify.sanitize(text, { ALLOWED_TAGS: ['b', 'i', 'em', 'strong', 'code', 'p', 'br'], ALLOWED_ATTR: [] });`

### ■ Critérios de Aceite
- [ ] O campo `component.group` é renderizado via JSX sem `dangerouslySetInnerHTML`.
- [ ] A sanitização de CVEs não permite injeção de links não autorizados sem atributos de segurança.
- [ ] Todos os testes unitários (`npm test`) continuam passando (74/74).
--- FIM ISSUE 1 ---
```

--- ISSUE 2 ---

Título: [Segurança] Bloquear pipeline de CI/CD na detecção de segredos (Gitleaks)

Labels: security, severity:medium, devsecops, ci/cd

■ Descrição do Problema

O job `gitleaks\_secret\_scan` em `.gitlab/ci/security.gitlab-ci.yml` está configurado com `allow\_failure: true`. Caso credenciais reais sejam acidentalmente commitadas no repositório, o pipeline do GitLab CI não é bloqueado, permitindo que a imagem Docker seja gerada e o deploy em produção seja executado.

■ Evidência no Código

Arquivo: `.gitlab/ci/security.gitlab-ci.yml` (Linhas 24-35)

```
``yaml
gitleaks_secret_scan:
  stage: security-scan
  image:
    name: zricethezav/gitleaks:latest
  entrypoint: [""]
  script:
    - gitleaks detect --source . --verbose --report-path gitleaks-report.json
  artifacts:
    name: gitleaks-report
    paths:
      - gitleaks-report.json
    expire_in: 1 week
  when: always
  allow_failure: true
````
```

### ■ Impacto

Publicação inadvertida de segredos em produção ou vazamento contínuo de credenciais no repositório sem alerta bloqueante.

### ■ Sugestão de Correção

Remover `allow\_failure: true` do job `gitleaks\_secret\_scan` e configurar arquivo `.gitleaksignore` para exceções pontuais documentadas caso necessário.

### ■ Critérios de Aceite

- [ ] `allow\_failure: true` removido do job `gitleaks\_secret\_scan`.
- [ ] Commit com segredo de teste falha no pipeline de CI/CD e aborta estágios subsequentes.
- [ ] Pipeline passa com sucesso em código limpo sem segredos.

--- FIM ISSUE 2 ---

## --- ISSUE 3 ---

**Título:** [Segurança] Endurecimento do Content-Security-Policy (CSP) no Nginx**Labels:** security, severity:low, infrastructure, nginx

## ### ■ Descrição do Problema

O arquivo `frontend/nginx.conf` possui a diretiva `unsafe-inline` na propriedade `style-src` do cabeçalho `Content-Security-Policy`.

## ### ■ Evidência no Código

Arquivo: `frontend/nginx.conf` (Linhas 30 e 161)

```
`add_header Content-Security-Policy "default-src 'self'; ... style-src 'self' 'unsafe-inline'
https://fonts.googleapis.com; ..." always;`
```

## ### ■ Impacto

Permite a injeção e execução de estilos CSS inline caso ocorra alguma vulnerabilidade secundária de injeção de HTML.

## ### ■ Sugestão de Correção

Remover a necessidade de `unsafe-inline` migrando estilos dinâmicos para classes utilitárias ou configurando hash sha256 específico para folhas de estilo inline essenciais.

## ### ■ Critérios de Aceite

- [ ] Diretiva `style-src` endurecida no `nginx.conf`.
- [ ] Nginx recarregado com configuração válida e sem erros no console do navegador.

--- FIM ISSUE 3 ---

## --- ISSUE 4 ---

**Título:** [Segurança] Limitação de tamanho de payload no Dropzone para prevenção de DoS client-side**Labels:** security, severity:low, frontend, performance

## ### ■ Descrição do Problema

Em `frontend/src/components/landing/Dropzone.tsx`, não há verificação do tamanho do arquivo antes de executar a leitura em memória com `FileReader` e o parsing JSON/XML.

## ### ■ Evidência no Código

Arquivo: `frontend/src/components/landing/Dropzone.tsx` (Linhas 47-74)

## ### ■ Impacto

Arquivos excessivamente volumosos (>100MB) podem exaurir a memória da aba do navegador, causando travamento da interface.

## ### ■ Sugestão de Correção

Adicionar validação `if (file.size > 50 \* 1024 \* 1024) { setError('Arquivo excede o tamanho máximo de 50MB.');

## ### ■ Critérios de Aceite

- [ ] Arquivos maiores que 50MB são rejeitados com mensagem clara antes do FileReader.
- [ ] Arquivos válidos normais continuam sendo processados sem impacto.

--- FIM ISSUE 4 ---